

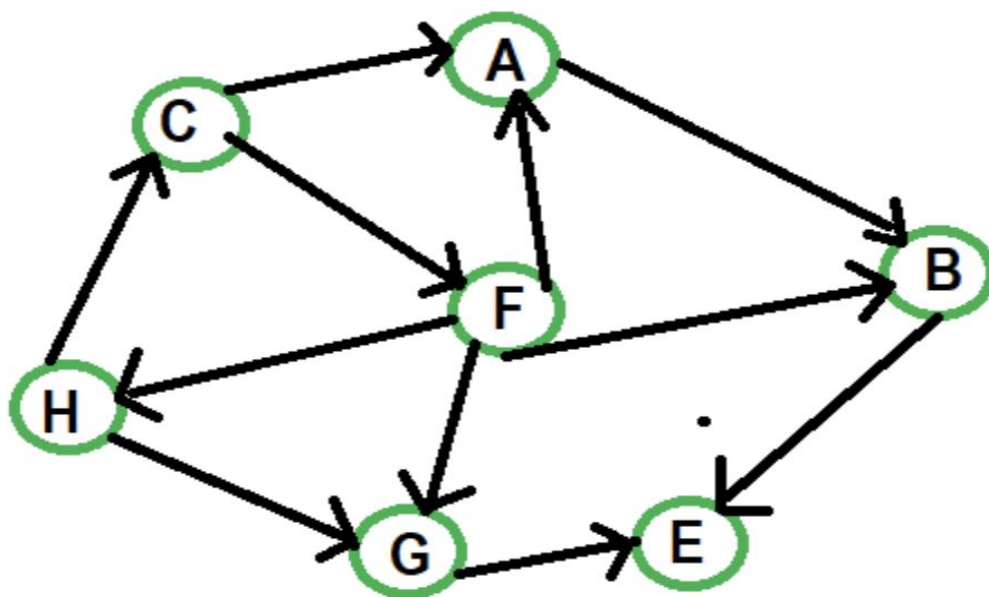
Apache Airflow - Summary

Definition:

Apache Airflow is an open-source platform that allows you to programmatically create, schedule, and monitor workflows. It is widely used by data engineers to orchestrate pipelines and manage complex data processes. With Airflow, you can easily visualize pipeline dependencies, track progress, view logs, manage code, trigger tasks, and monitor their success or failure. It is a powerful and reliable tool for handling large-scale data pipelines that deliver datasets to business intelligence applications and machine learning models requiring vast amounts of data. Designed for batch-oriented workflows, Airflow makes building, scheduling, and monitoring data pipelines both efficient and seamless.

Working of Airflow:

Workflow refers to the process of achieving some goal. They always have an end goal which could be something like creating visualizations for some data as given here. Directed Acyclic Graphs (abbreviated as DAG) are used to represent the workflow.



Components of Airflow:

1. Directed Acyclic Graph (DAG)

A DAG is the **core concept** in Airflow.

It represents a workflow, defined as a set of tasks with dependencies and execution order.

Ensures no cycles are present (tasks don't depend on each other in a loop).

2. Task

A task is a **single unit of work** in Airflow.

Each task is an instance of an operator (e.g., PythonOperator, BashOperator, SQL operators).

Tasks can be executed independently or in sequence depending on dependencies.

3. Operator

Operators define the **type of work** a task will perform.

Examples:

PythonOperator → runs Python code

BashOperator → runs shell scripts

SQLOperator → runs SQL queries

Think of operators as **templates** for tasks.

4. Scheduler

The scheduler is responsible for **scheduling jobs** (tasks in DAGs) to run at the specified time or intervals.

It decides what tasks need to be executed and hands them over to the executor.

5. Executor

The executor is the component that **actually runs the tasks**.

Works with workers (local or distributed) to execute tasks in parallel.

Types of executors: LocalExecutor, CeleryExecutor, KubernetesExecutor, etc.

6. Worker

Workers are the **machines or processes** that carry out the actual execution of tasks.

They receive tasks from the executor and run them.

7. Web UI (Airflow UI)

A rich web-based interface for monitoring and managing workflows.

Provides visualization of DAGs, task progress, logs, retries, and execution history.

8. Metadata Database

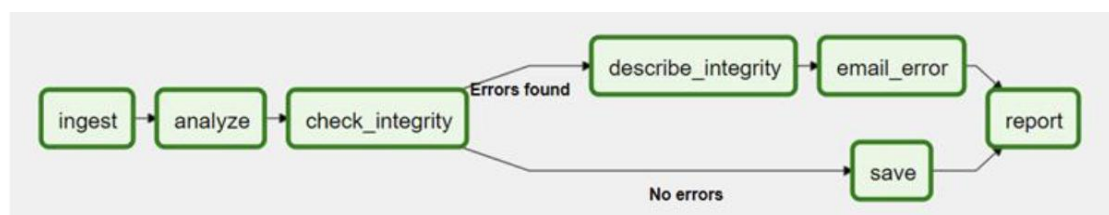
A relational database (like PostgreSQL or MySQL) that stores all the information about:

DAGs

Task states (success, failed, running, skipped)

User permissions

Acts as the **central source of truth** for Airflow.



Benefits of Apache Airflow

- **Open Source & Free** – No licensing costs; widely supported by the community.
- **Scalable** – Can handle simple to very complex workflows; supports parallel execution.
- **Extensible** – Custom operators, sensors, and plugins can be added easily.
- **Dynamic Pipelines** – Workflows are written in Python, making them flexible and programmable.
- **Clear Visualization** – Web UI shows DAGs, dependencies, progress, logs, and status.
- **Robust Scheduling** – Supports cron-like scheduling and event-based triggers.
- **Retry & Error Handling** – Automatic retries, alerts, and logging when tasks fail.
- **Integration Friendly** – Works well with cloud services (AWS, GCP, Azure), databases, and big data tools.
- **Monitoring & Alerts** – Built-in monitoring with email/SMS/Slack alerts.
- **Community Support** – Large user base, continuous improvements, and active contributors.

In short: Airflow makes managing, scaling, and monitoring workflows easier, reliable, and developer-friendly.

Components of Airflow

- **Dynamic**
Airflow enables pipelines to be dynamically created using Python code. This makes workflows flexible and easily configurable.

- **Extensible**

Airflow is highly extensible, allowing users to build custom operators.
This lets workflows adapt to different environments and requirements.

- **Elegant**

Pipelines in Airflow are simple, lean, and easy to understand.
The code is explicit, making workflows clear and maintainable.

- **Scalable**

Airflow has a modular architecture that supports scaling.
It uses a message queue to handle communication efficiently.